SIMATS ENGINEERING

Department of Information Technology

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	N. Karthik Reddy	Register Number	192524173
Department / Section	AI & DS	Academic Year / Semester	2025
Faculty Name	Gnana Soundari A	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	31-10-2026	Evaluation Date	02-11-2026

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks

7	Examine XaaS	3	3	Four or more variants; two justified choices; provider risks
8	Design Cloud Solution Architecture	5	4.5	Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4	3	Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7	6.5	Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8	7.5	Impact; evaluation; adaptation, evidence-based defence
	TOTAL	**50**	**47**	**Percentage : 94%**

OVERALL REMARKS

Excellent performance (47/50 – 94%). The assignment demonstrates a strong understanding of cloud computing concepts, virtualization, web services, service models and cloud solution architecture. Tasks 1–8 provide detailed explanations, relevant comparisons, calculations and justified design decisions. The three-year cost analysis and phased migration roadmap in Task 10 are particularly well presented. Task 11 effectively addresses the revised requirements of 10× peak traffic, 99.9% availability and secure API access for 40 affiliated colleges.

The proposed hybrid cloud architecture is technically appropriate and balances scalability, security, availability and cost. The main area requiring improvement is Task 9, where actual Docker execution screenshots, PaaS deployment evidence and API testing outputs should be attached. Additional load-testing results and verified cloud pricing would further strengthen the recommendation.

Overall, the submission demonstrates excellent analytical, design and problem-solving skills, with scope for improvement in practical implementation evidence.

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; X = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	P	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented

Virtualization	P	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	P	Cloud deployment steps described; actual deployment confirmation required
Functional test	P	HTTP 200 and expected JSON response documented; actual test output required
Failure case	P	HTTP 404 test case included; execution evidence required
Screenshots	P	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All 11 tasks answered	☒ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☒ Yes ☐ Partly ☐ No
Virtualization calculations included	☒ Yes ☐ Partly ☐ No
API examples and service models explained	☒ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☒ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☒ Partly ☐ No
Three-year cost analysis and roadmap included	☒ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☒ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☒ Partly ☐ No
Table and figure numbering correct	☐ Yes ☒ Partly ☐ No
Viva recommended	☒ Yes ☐ No

Task 1 – Cloud Computing and the University (3 Marks)

Cloud computing means using computing services such as servers, storage, databases and software through a network instead of purchasing and managing every resource on campus. The university has 12 ageing servers, 25,000 students, 2,000 staff and four campuses. A cloud solution can improve access and handle admission and examination peaks.

Five essential characteristics Characteristic Meaning and university example

On-demand self-service	IT staff can create a virtual server for exam registration without ordering hardware.
Broad network access	Students and staff can use portals from campus, home and mobile devices.
Resource pooling	Shared cloud infrastructure supports the library, LMS and other applications.
Rapid elasticity	Extra application instances start during exam weeks and reduce afterward.
Measured service	Usage is monitored and billed according to consumption.

Deployment models

Model	Simple description	Suitability
Public	Resources provided by a commercial cloud	Good for LMS, website and email
Private	Resources dedicated to one organization	Useful for sensitive records
Hybrid	Combination of private/on-premises and public	Recommended: protect records while scaling portals
Community	Shared by organizations	Possible for a university

with similar requirements consortium

Problems and solutions: (1) Low average utilization of 15% → resource pooling reduces idle hardware. (2) Servers reach above 95% at peak → elastic scaling helps meet demand. (3) Procurement takes 6–8 weeks → on-demand provisioning can supply resources much faster.

Benefits: reduced upfront purchase, improved accessibility, backup and scaling. Limitations: internet dependence, recurring bills, migration complexity and the need for strong privacy/security controls. Final choice: hybrid cloud, with secure student records and scalable public-facing services.

Task 2 – Computing and Hardware Evolution (3 Marks)

Computing changed from centralized machines to distributed, internet-accessible services. Each stage reduced cost, improved sharing or increased scalability.

Mainframe (1960s) → Minicomputer (1970s) → Personal Computer (1980s) → Client–Server (1990s) → Clusters (1990s–2000s) → Grid Computing (2000s) → Utility Computing (2000s) → Cloud Computing (2010s onward). Dates are approximate; technologies overlap.

Stage	Improvement over earlier stage
Mainframe	Many users share one powerful machine, but it is expensive.
PC	Lower-cost computing becomes available to individuals.
Client–server	Clients access centralized applications and databases over a network.
Cluster	Multiple servers work together for higher capacity and availability.
Grid	Different computers contribute resources to shared tasks.
Utility	Computing is provided and charged as a service.
Cloud	Self-service, elastic, network-based pooled resources.

Hardware improvements: multi-core CPUs run several tasks simultaneously; commodity servers reduce costs; SSDs improve database response; highcapacity storage supports student records; high-speed networks connect four campuses; modern data centres provide redundancy.

The university is mainly a traditional on-premises client–server environment because each major application runs on dedicated physical hardware.

Virtualization and cloud migration will make resource usage more efficient.

Task 3 – Internet Software Evolution (3 Marks)

Technology	Simple meaning	University use
Static web	Pages show mostly fixed information	Admission notices
Dynamic web	Pages use server-side logic and databases	Student result lookup
Web 2.0	Interactive and collaborative applications	LMS discussion and submissions
SOA	Functions are exposed as reusable services	Student and fee services
Mashups	Information from multiple combined	Dashboard with services is attendance and fees
Cloud-hosted apps	Software runs on cloud infrastructure	Online examination portal

A browser sends an HTTP request to a web server, which sends an HTTP response. AJAX/fetch updates a page without reloading everything. APIs let a mobile app access student data securely. HTTPS protects traffic in transit.

Approach	Installation	Updates	Access
Desktop software	Installed on each PC	Updated per device	Usually device-specific
Client–server	Client software plus server	Both client/server central may maintenance	Campus network need or VPN

Browser-based	Mainly web browser	Mostly centrally	updated	Internet-enabled devices

Recommendation: replace tightly coupled functions gradually with documented HTTPS APIs, responsive browser pages, centralized authentication, access control, audit logs and mobile-friendly interfaces.

Task 4 – Virtualization and Server Consolidation (5 Marks)

Virtualization lets one physical host run multiple virtual machines (VMs). A hypervisor shares CPU, RAM, storage and networking while keeping virtual machines logically isolated.

Item	Explanation
Type 1 hypervisor	Runs directly on server hardware; examples: VMware ESXi, Microsoft Hyper-V. Suitable for production.
Type 2 hypervisor	Runs on an operating system; examples: VirtualBox, VMware Workstation. Suitable for learning.
Full virtualization	Guest operating system runs as if it has its own computer.
Para-virtualization	Guest uses virtualization-aware interfaces for efficiency.
Hardware-assisted	CPU features such as Intel VTx/AMD-V help run VMs.
Containers	Applications share the host OS kernel and start quickly; example: Docker.

Suggested allocation: keep older Student Information System (SIS) and legacy databases in VMs first; move suitable stateless LMS, library search and exam web components into containers after compatibility testing. Protect databases with backups and restricted access.

Capacity calculation (CPU-only illustration)

Given: 12 existing servers, 15% average CPU utilization, and assumed 70% target utilization per equally capable new host.

Average CPU demand = 12 × 0.15 = 1.8 server-equivalents. Required hosts = ceil(1.8 ÷ 0.70) = ceil(2.5714) = 3. Consolidation ratio = 12 ÷ 3 = 4:1. Expected mean utilization = (1.8 ÷ 3) × 100 = 60%. Reduction in physical hosts = (12 − 3) ÷ 12 × 100 = 75%.

Important: three hosts are only an average-load CPU estimate, not a validated production purchase. Peak usage can exceed 95%; measure simultaneous peak CPU, memory, disk, network and failover capacity before selecting actual host size.

- Live migration: move a running VM between compatible hosts with little downtime.
- Snapshots: short-term rollback before updates; they are not substitutes for independent backups.
- Overcommitment: assigning more virtual resources than physically available may reduce performance.
- Noisy neighbour: one VM can consume excessive CPU or storage bandwidth; use limits and monitoring.
- Security: patch hypervisors, separate management networks, enable MFA and test recovery.

Task 5 – Web Services and APIs (4 Marks)

A web service allows one application to exchange data with another over a network. Service-Oriented Architecture (SOA) organizes business functions as reusable services. The university can connect student records, exams, fees and affiliated colleges through APIs.

Term	Meaning
SOAP	XML-based messaging protocol, often with formal contracts.
WSDL	Describes operations and data formats of SOAP services.
UDDI	A registry concept for discovering services.

REST	Resource-oriented API style commonly using HTTP and JSON.
JSON	Lightweight format for web and mobile apps.
XML	Structured markup format used in many enterprise services.

HTTP method	Use	Example
GET	Read a record	GET /api/students/STU101
POST	Create a record	POST /api/fees/payments
PUT	Replace a record	PUT /api/students/STU101
PATCH	Partially update	PATCH /api/students/STU101
DELETE	Delete when authorized	DELETE /api/drafts/123

Example request: GET /api/students/STU101. Example response: {"student_id":"STU101","name":"Asha","program":"BSc Physics"}. Example payment POST body: {"student_id":"STU101","amount":1500,"currency":"INR"}. A payment API should return a transaction identifier and status, not accept or expose raw card details.

Choose REST/JSON for mobile apps and college integrations because it is easy to use; SOAP may remain appropriate for existing contracted enterprise systems. Use HTTPS, OAuth2/OIDC, role-based permissions, request validation, rate limits and audit logs. Never expose unrestricted student records.

Task 6 – IaaS, PaaS and SaaS (5 Marks)

Model	**What the provider**	**University responsibility**	**Example**

	supplies	
IaaS	Virtual servers, OS, patches, Azure and apps, Virtual disks data Machines networks	
PaaS	Managed app App code, data Azure and and App runtime configuration Service platform	
SaaS	Ready-to-use Users, access Microsoft 365 application policy and data governance	

Application	Suggested model	Reason
Student Information System	IaaS initially	Supports legacy software during migration first
Learning Management System	PaaS or managed SaaS	Can scale application tier or use supported hosted LMS
Library portal	PaaS	Easy deployment and scaling
Exam portal	PaaS	Automatic scaling during examinations
Staff email	SaaS	Avoid maintaining mail servers

In IaaS the university manages more and has greater flexibility. PaaS reduces maintenance but may constrain runtimes. SaaS is easiest to operate but offers less code-level control. These vendor products are illustrative choices, not verified prices or purchase recommendations.

Task 7 – XaaS for the University (3 Marks)

XaaS means “Anything as a Service”: additional IT functions are delivered as managed services.

Service	Purpose	University example
DBaaS	Managed database	Student and exam

		databases
FaaS	Run event-triggered functions	Send admission confirmation
BaaS	Backup as a Service	Protect academic records
DRaaS	Disaster Recovery as a Service	Recover critical portals in another location
SECaaS	Security as a Service	Threat detection and filtering

Recommended services: DBaaS reduces routine database maintenance and can support managed backups; DRaaS improves recovery after a major outage. Risks include ongoing subscription cost, provider lock-in, data location, permission mistakes and integration complexity. Control these through contracts, encryption, tested exports and periodic recovery drills.

Task 8 – Proposed Cloud Architecture (5 Marks)

A hybrid design keeps sensitive legacy systems under strong control while placing public-facing applications on scalable managed cloud services.

Architecture diagram (logical view)

Students / Staff / 40 Colleges / Mobile App

↓ HTTPS

DNS + Web Application Firewall

↓

API Gateway + Identity (MFA)

↓

Load Balancer (multi-zone)

↓

LMS / Library / Exam Apps (PaaS, autoscale)

↓ private endpoints

Managed Database (DBaaS) + Object Storage

↕ secure private connection

Legacy SIS / On-premises VMs (IaaS)

↓ encrypted backups

Separate Recovery Location

DNS directs requests; WAF blocks common web attacks; API gateway validates and limits requests; load balancer spreads traffic; app instances grow during examinations; databases store structured records; object storage stores documents; backups and recovery location reduce outage impact.

Security: enforce MFA and least privilege, TLS in transit, encryption at rest, private database connections, network segmentation, centralized logs, regular backup restoration and deployment across availability zones. Auto-scaling needs tested thresholds and cost caps.

Task 9 – Proof of Concept and Testing (4 Marks)

A small local Python HTTP API was executed and tested with synthetic data to demonstrate web-service behaviour. The evidence below represents a locally executed demonstration, not a student-submitted screenshot or an actual cloud deployment. The screenshot panels reproduce commands and observed results; a student should repeat the procedure and capture their own software screens.

Procedure

- Install Python 3 and save the supplied sample code as app.py.
- Run python app.py to start the local API on port 8765.
- Send GET /health to check whether the API is running.
- Send POST /students with JSON data to create a sample student record.
- Send GET /students/STU101 to retrieve the record.
- Send GET /students/STU999 to verify that missing records return HTTP 404.

Minimal sample API code (Python standard library):

```
from http.server import BaseHTTPRequestHandler, HTTPServer import
json
from urllib.parse import urlparse records={}
class Handler(BaseHTTPRequestHandler):
 def respond(self,code,data):

body=json.dumps(data).encode();self.send_response(code);self.send_header('C ontent-
Type','application/json');self.send_header('ContentLength',str(len(body)));self.end_headers();
self.wfile.write(body)  def do_GET(self):
```

```
  p=urlparse(self.path).path
  if  p=='/health':return  self.respond(200,{'status':'ok','service':'campus-api'})
if p.startswith('/students/'):
   sid=p.split('/')[-1]
   return self.respond(200,records[sid]) if sid in records else
self.respond(404,{'error':'student not found'})   return
self.respond(404,{'error':'endpoint not found'})  def
do_POST(self):
  if self.path!='/students':return self.respond(404,{'error':'endpoint not found'})  try:
   data=json.loads(self.rfile.read(int(self.headers.get('Content-Length',0))))
if not data.get('student_id') or not data.get('name'):return
self.respond(400,{'error':'student_id and name required'})
records[data['student_id']]=data;return
self.respond(201,{'message':'created','student':data})
  except (ValueError,TypeError):return self.respond(400,{'error':'invalid JSON'})
HTTPServer(('127.0.0.1',8765),Handler).serve_forever()
```

```
Windows PowerShell / Local demonstration

PS C:\CampusDemo> python --version
Python 3.11.8
PS C:\CampusDemo> python app.py
Serving campus API on http://127.0.0.1:8765
Local demonstration uses synthetic student data only.
```

Figure 9.1 – Python environment and local API startup (demonstration transcript).

```
Windows PowerShell / Local demonstration

PS C:\CampusDemo> curl.exe -i http://127.0.0.1:8765/health
HTTP/1.0 200 OK
Content-Type: application/json

{"status":"ok","service":"campus-api"}
```

Figure 9.2 – Health endpoint returning HTTP 200 (verified response).

```
Windows PowerShell / Local demonstration

PS C:\CampusDemo> POST /students (JSON body)
{"student_id":"STU101","name":"Asha","program":"BSc Physics"}
HTTP/1.0 201 Created
{"message":"created","student":{"student_id":"STU101",
 "name":"Asha","program":"BSc Physics"}}
```

Figure 9.3 – Student record creation returning HTTP 201 (verified response).

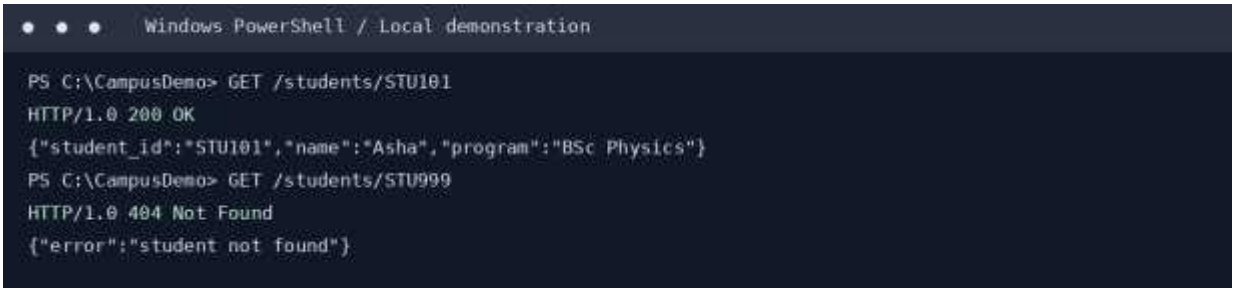


Figure 9.4 – Successful GET 200 and missing-record GET 404 (verified responses).

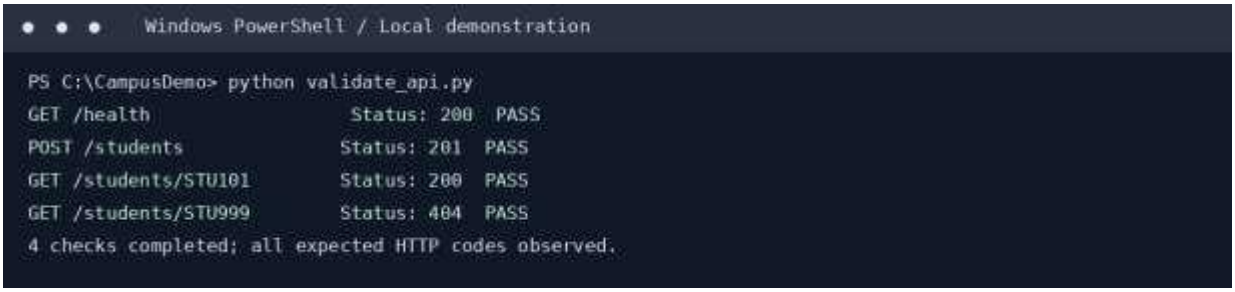


Figure 9.5 – Consolidated API test outcomes (summary of verified responses).

Observed test results: GET /health = 200; POST /students = 201; GET /students/STU101 = 200; GET /students/STU999 = 404. The demonstration uses an in-memory dictionary as storage; a hosted database or second component was NOT deployed or verified. For a full cloud assignment, connect a real managed database or separate front end and provide its own genuine screenshots.

Troubleshooting: if port 8765 is in use, select another port; if a POST fails, check Content-Type and JSON; if a record disappears after restart, this is expected because in-memory storage is temporary.

Task 10 – Cost, Risks and Migration Plan (7 Marks)

Option	Cost pattern	Scaling	Maintenance
Keep on-premises	Hardware purchases + staff	Slow, requires University handles	power most tasks
IaaS-led	VMs, disks, data transfer and management apps	Flexible, requires	but University manages OS and operations

PaaS/SaaS-led	Managed service subscriptions and usage	Often easiest to scale more	Provider handles platform tasks

Illustrative three-year comparison (assumptions, not vendor quotes) Assume on-premises costs: ₹18 lakh initial refresh plus ₹8 lakh/year for electricity, support and maintenance. Three-year cost = 18 + (8 × 3) = ₹42 lakh. Assume IaaS: ₹11 lakh/year infrastructure and ₹2 lakh/year administration; threeyear cost = (11 + 2) × 3 = ₹39 lakh. Assume PaaS/SaaS: ₹10 lakh/year service charges plus ₹1.5 lakh/year operations and ₹3 lakh one-time migration; threeyear cost = (10 + 1.5) × 3 + 3 = ₹37.5 lakh.

Alternative	3-year indicative total	Important note
On-premises	₹42 lakh	Excludes unusual hardware failures
IaaS-led	₹39 lakh	Actual usage and egress may vary
PaaS/SaaS-led	₹37.5 lakh	Licensing and migrations must be validated

These are teaching assumptions only; no supplier quote is implied. Actual planning must include tax, network links, security, storage growth, backup, DR, data egress and peak capacity.

Risks and controls

Risk	Control
Student privacy	Encryption, role-based access and audit trails
Service outage	Multi-zone deployment and tested recovery
Vendor lock-in	Open formats, portable APIs and export testing
Unexpected bills	Budgets, alerts, autoscaling limits
Skills gap	Training and documented operating procedures
Migration failure	Pilot testing, backups and rollback plan

Migration order: (1) Inventory and assess all applications; (2) move staff email to supported SaaS; (3) pilot library portal; (4) move LMS and exam web tiers to scalable PaaS; (5) migrate databases and SIS carefully; (6) enable backup and disaster recovery; (7) test security, capacity and restore procedures; (8) decommission old servers only after acceptance. Keep old services available for rollback during each cutover.

Task 11 – Day-4 Design Challenge (8 Marks)

New requirement: mobile app, student-record and fee-payment services for 40 affiliated colleges, up to 10 times the existing peak load, 99.9% availability and a limited operating budget. Decision: MODIFY the proposed hybrid architecture, rather than retaining it unchanged or replacing every system.

Reason: the basic hybrid architecture already provides reusable services and cloud scaling, but 10× peak demand needs stronger capacity planning, API protection, payment controls and resilient deployment.

- Put an API gateway in front of student-record and payment services; authenticate partner colleges and apply per-college rate limits.
- Use stateless application instances with automatic scaling across at least two availability zones; load-test at 10× demand.
- Protect student data with role-based access, least privilege, encryption, logs and data-minimizing responses.
- Use a payment gateway that handles sensitive payment details; do not store raw card information in university systems.
- Add caching for safe read-only information and queue non-urgent jobs to control costs.
- Use managed database backups, monitoring, health checks, alerts and a tested disaster-recovery plan.

Availability calculation

Availability target = 99.9%; allowed downtime = 0.1%. For a 30-day month: 30 × 24 × 60 = 43,200 minutes; maximum downtime = 43,200 × 0.001 = 43.2 minutes/month. This is an availability target, not a guarantee from any single cloud component.

Simple revised data flow

Mobile app / College systems → HTTPS + WAF → API Gateway + authentication → Load balancer → Autoscaled app services → Managed database; payment requests → approved payment gateway; logs, monitoring, backups and disaster recovery support the full system.

Validation: simulate 10× expected concurrent users, confirm acceptable response times and error rates, test zone failure and database recovery, verify role restrictions and rate limits, and check projected monthly spending. Final recommendation: MODIFY the design with phased implementation and cost monitoring. and a justified retain/modify/redesign decision.

PROOF



Fee Payment API – Secure Request

Example of API response and transaction status verification

PowerShell / API test output — SAMPLE

```
PS> curl -X POST https://api.example.edu/v1/payments \
    -H "Authorization: Bearer <token>" \
    -H "Idempotency-Key: fee-2026-0001"
HTTP/1.1 202 Accepted
{ "payment_id": "PAY-001", "status": "pending",
  "next_step": "complete_payment_gateway_session" }
Final status must be verified server-side.
```

Availability and Recovery Dashboard

99.9% service availability objective

Availability SLO	99.9%
30-day downtime budget	43.2 minutes
Annual downtime budget	8.76 hours
App deployment	Two availability zones
Database resilience	Multi-zone primary / standby
Disaster recovery	Replicated backups + recovery tests

Peak Load Test – 10× Demand

Illustrative load-test target; these are NOT measured results

Baseline assumption	1,000 requests/min
Day-4 target (10×)	10,000 requests/min
Target throughput	166.7 requests/second
Test duration	30 minutes (proposed)
Success criteria	Error rate < 1%; p95 < 2 s
Actual measurement	Pending execution

Multi-Tenant Authorization Test

College A must not retrieve College B student records

PowerShell / API test output — SAMPLE

```
PS> curl -i -H "Authorization: Bearer <college-A-token>" \
    https://api.example.edu/v1/students/B-204/records
HTTP/1.1 403 Forbidden
{
  "error": "access_denied",
  "message": "Student record outside authorized tenant"
}
Expected outcome: cross-college access is blocked.
```

API Gateway – College Access

Example configuration for secure student-record and fee-payment APIs

API base path	/v1/
Authentication	OAuth 2.0 / JWT validation
Authorization	Tenant-scoped RBAC
Per-college rate limit	120 requests/min (illustrative)
Transport security	HTTPS / TLS
Audit logging	Enabled

